

PEC 2: Juegos

Presentación

Segunda PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas informadas, su formalización y estrategias relacionadas.

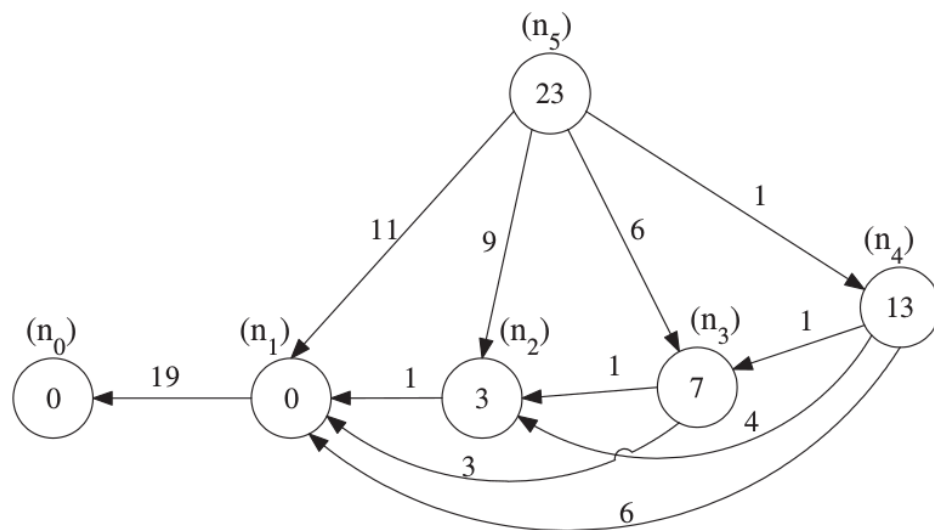
Descripción de la PEC/práctica a realizar

Esta PEC tiene como objeto la búsqueda con heurísticos y el algoritmo A*, y profundizar en un concepto que no aparece en los materiales, el de función heurística *consistente*, y las implicaciones que esto tiene para la implementación del A*.

Según la literatura, un heurístico sobre un grafo es **consistente** si es un heurístico admisible con la siguiente propiedad: Para todo par de nodos de un grafo x e y , si hay un camino del nodo x al nodo y , entonces $h(x) \leq \text{coste}(x, y) + h(y)$, donde $\text{coste}(x, y)$ es el coste de ir de x a y . Judea Pearl demostró que podemos restringir y a ser un vecino de x para dar una definición más intuitiva, aunque equivalente: El hecho de ir de un nodo a un vecino suyo no tiene que hacer decrecer h más de lo que crece g . Diremos que un heurístico se **inconsistente** cuando no es consistente.

Fijémonos que la admisibilidad forma parte de la definición de consistencia, por lo tanto, el hecho que un heurístico sea consistente implica, por definición, que es admisible y que A^* encontrará la solución óptima si utilizamos este heurístico. También observamos que si $\text{consistente} \Rightarrow \text{admisible}$, entonces $\text{no admisible} \Rightarrow \text{inconsistente}$.

Alberto Martelli investigó precisamente este problema, y definió una familia de grafos G_i , donde $\{i \geq 3\}$, llamados *Grafos de Martelli* con determinadas propiedades especiales. Dado el **grafo de Martelli G_5** :



donde el número sobre las aristas es el coste de ir de un nodo a otro y el heurístico es el número dentro de cada nodo. Así, $h(n_5) = 23$, $h(n_4) = 13$, etc. El objetivo es encontrar el camino de menor coste para ir de n_5 a n_0 .

Pregunta 1 (20%): Estudiar la admisibilidad y la consistencia del heurístico definido en el grafo G_5 del dibujo. Cualquier respuesta debe justificarse (si se afirma algo hay demostrarlo, aunque sea por enumeración de todas las posibilidades, si se niega hay que aportar un contra-ejemplo).

Solución: Demostrar que h es admisible es sencillo, ya que el coste óptimo de llegar de cualquier nodo a n_0 no puede ser inferior a 19, es decir, $h^*(n_j) \geq 19$, para todo j entre 1 y 5. Esto se debe a la arista que va de n_1 a n_0 . Y todos los heurísticos son menores que 19: $h(n_j) < 19 \leq h^*(n_j)$, para todos los nodos excepto n_5 . El camino óptimo para n_5 tiene coste 23, como veremos después al aplicar A^* , pero es bastante evidente. Como $23 = h(n_5) \leq h^*(n_5) = 23$, h es admisible. La inconsistencia de h se ve ya que $13 = h(n_4) \not\leq \text{coste}(n_4, n_3) + h(n_3) = 1 + 7$. Hay más inconsistencias.

Vamos ahora a profundizar en la implementación de A^* y la relación con la consistencia del heurístico. Se puede practicar en la aplicación manual del algoritmo A^* estudiando las soluciones de las PEC2 de los dos semestres del curso pasado (en las carpetas **2017_1 set17-feb18** y **2017_2 feb18-jul18** del paquete de PECs resueltas). Fijémonos que allí utilizamos dos listas de nodos abiertos y cerrados, revisando costes en caso de mejorarlos. En cambio, si se consulta el pseudo-código que aparece en la Wikipedia, en la entrada que corresponde al A^* : en.wikipedia.org/wiki/A*_search_algorithm, se encontrará un algoritmo donde, al expandir un nodo, si alguno de sus vecinos está en el conjunto de cerrados lo ignoramos, sin revisar para nada sus costes.

Consideraremos dos algoritmos A^* diferentes:

A^ completo:* Al considerar los sucesores del nodo actual, si encontramos uno que ya está en el conjunto de cerrados calculamos su coste y, si mejora el coste que tenía cuando se le colocó en el conjunto de cerrados, sacamos el sucesor del conjunto cerrados y lo volvemos a poner en el conjunto abiertos con el coste revisado (fijémonos que esto es un poco diferente a lo que hacemos en los ejemplos mencionados de las PECs del curso pasado, en las PECs revisamos los sucesores del sucesor, los sucesores de los sucesores, etc. lo que proponemos aquí es más sencillo y equivalente).

A^ simplificado:* Al considerar los sucesores del nodo actual, si encontramos uno que ya está en el conjunto de cerrados lo ignoramos. Este es el caso del pseudo-código mencionado que se puede encontrar en la Wikipedia.

Pregunta 2 (20%): ¿Por qué razón en algunos casos podemos simplificar la implementación del A^* y no revisar los costes de los nodos dentro del conjunto de cerrados (como se hace en la Wikipedia)? ¿Qué tiene que ver con la consistencia del heurístico?

Solución: Si miramos en la misma Wikipedia, vemos el *Remark* que hay bajo el pseudo-código: *the above pseudocode assumes that the heuristic function is monotonic (or consistent, see below), which is a frequent case in many practical problems, such as the Shortest Distance Path in road networks. However, if the assumption is not true, nodes in the closed set may be rediscovered and their cost improved. In other words, the closed set can be omitted (yielding a tree search algorithm) if a solution is guaranteed to exist, or if the algorithm is adapted so that new nodes are added to the open set only if they have a lower f value than at any previous iteration.* Por lo tanto, si el heurístico es consistente, el *primer* camino que encuentra el A^* es el mejor, y por eso podemos ignorar los nodos ya visitados (que están en el conjunto de nodos cerrados).

Pregunta 3 (40%): Utiliza el algoritmo A* completo y el A* simplificado para encontrar, manualmente, el camino mínimo para ir de n_5 a n_0 en G_5 . ¿El resultado es el mismo? ¿Cuántos nodos has expandido en cada caso?

Solución: Si aplicamos a mano el pseudo-código de la Wikipedia, o cualquier versión del A* donde se ignoren los nodos cerrados, obtenemos:

Open: (5,23.0)

Closed:

Open: (1,11.0) (2,12.0) (3,13.0) (4,14.0)

Closed: (5,23.0)

Open: (0,30.0) (2,12.0) (3,13.0) (4,14.0)

Closed: (1,11.0) (5,23.0)

Open: (0,30.0) (3,13.0) (4,14.0)

Closed: (1,11.0) (2,12.0) (5,23.0)

Open: (0,30.0) (4,14.0)

Closed: (1,11.0) (2,12.0) (3,13.0) (5,23.0)

Open: (0,30.0)

Closed: (1,11.0) (2,12.0) (3,13.0) (4,14.0) (5,23.0)

5->1 11.00 1->0 19.00

Y la respuesta, como observamos, tiene coste 30 y por tanto es incorrecta. En cambio, utilizando el A* completo, sin restricciones, obtenemos un desarrollo de la búsqueda mucho mayor, pero correcto:

Open: (5,23.0)

Closed:

Open: (1,11.0) (2,12.0) (3,13.0) (4,14.0)

Closed: (5,23.0)

Open: (0,30.0) (2,12.0) (3,13.0) (4,14.0)

Closed: (1,11.0) (5,23.0)

Open: (0,30.0) (1,10.0) (3,13.0) (4,14.0)

Closed: (2,12.0) (5,23.0)

Open: (0,29.0) (3,13.0) (4,14.0)

Closed: (1,10.0) (2,12.0) (5,23.0)

Open: (0,29.0) (1,9.0) (2,10.0) (4,14.0)

Closed: (3,13.0) (5,23.0)

Open: (0,28.0) (2,10.0) (4,14.0)

Closed: (1,9.0) (3,13.0) (5,23.0)

Open: (0,28.0) (1,8.0) (4,14.0)

Closed: (2,10.0) (3,13.0) (5,23.0)

Open: (0,27.0) (4,14.0)

Closed: (1,8.0) (2,10.0) (3,13.0) (5,23.0)

Open: (0,27.0) (1,7.0) (2,8.0) (3,9.0)

Closed: (4,14.0) (5,23.0)

Open: (0,26.0) (2,8.0) (3,9.0)

Closed: (1,7.0) (4,14.0) (5,23.0)

Open: (0,26.0) (1,6.0) (3,9.0)

Closed: (2,8.0) (4,14.0) (5,23.0)

Open: (0,25.0) (3,9.0)

Closed: (1,6.0) (2,8.0) (4,14.0) (5,23.0)

Open: (0,25.0) (1,5.0) (2,6.0)

Closed: (3,9.0) (4,14.0) (5,23.0)

Open: (0,24.0) (2,6.0)

Closed: (1,5.0) (3,9.0) (4,14.0) (5,23.0)

Open: (0,24.0) (1,4.0)

Closed: (2,6.0) (3,9.0) (4,14.0) (5,23.0)

Open: (0,23.0)

Closed: (1,4.0) (2,6.0) (3,9.0) (4,14.0) (5,23.0)

5->4 1.00 4->3 1.00 3->2 1.00 2->1 1.00 1->0 19.00

Se recuerda que en el Módulo 2 hay una implementación en Common Lisp de un sistema de búsqueda general que, con pocas adaptaciones, se puede transformar en diferentes tipos de búsquedas. Así, el código del sistema general está descrito en los apartados 1.1.3, 2.1 y tiene la búsqueda en anchura definida en el apartado 3.1.1, la búsqueda en profundidad en el apartado 3.2, la búsqueda en profundidad limitada en el apartado 3.2.1 y la búsqueda A* a partir del apartado 4.3.2. Para ilustrar el código, éste está aplicado al ejemplo del rompecabezas lineal.

En esta PEC2 tenéis el código correspondiente al A* en un fichero **busqueda-pac2.lisp** que adjuntamos con la PEC2. Allí el sistema del Módulo 2 está implementado (con un par de modificaciones indicadas en el código, para hacer más comprensible el resultado) con énfasis en la búsqueda A*.

Pregunta 4 (20%): ¿Qué versión del A* está implementada en Common-Lisp? ¿La versión simplificada o la completa? En el fichero **busqueda-pac2.lisp** está el grafo G_5 a medio definir, sólo hay que añadir la función de coste y el heurístico. Fíjense en los ejemplos de las carpetas **2017_1 set17-feb18** y **2017_2 feb18-jul18** del paquete de PECs resueltas para tener claro el formato en el que deben implementarse las funciones.

Solució: Definiremos el grafo G_5 en el formato pertinente:

```
;;;;;;;;; Definición del grafo:
;;;;;;;;; Un operador por cada arista del grafo
;;;;;;;;; DEFINICION DE G5
;; ...
;;;;;;;;; Funció de cost
(defun coste (estado1 estado2)
  (cond ((and (equal estado1 'A) (equal estado2 'B)) 1)
        ((and (equal estado1 'A) (equal estado2 'C)) 6)
        ((and (equal estado1 'A) (equal estado2 'D)) 9)
        ((and (equal estado1 'A) (equal estado2 'E)) 11)
        ((and (equal estado1 'B) (equal estado2 'C)) 1)
        ((and (equal estado1 'B) (equal estado2 'D)) 4)
        ((and (equal estado1 'B) (equal estado2 'E)) 6)
        ((and (equal estado1 'C) (equal estado2 'D)) 1)
        ((and (equal estado1 'C) (equal estado2 'E)) 3)
        ((and (equal estado1 'D) (equal estado2 'E)) 1)
        ((and (equal estado1 'E) (equal estado2 'F)) 19)
        (t 100)))
```

;;;;;;;;; Función heurística

```
(defun heuristica (estado)
  (cond ((equal estado 'A) 23)
        ((equal estado 'B) 13)
        ((equal estado 'C) 7)
        ((equal estado 'D) 3)
        ((equal estado 'E) 0)
        ((equal estado 'F) 0)
        (t 100)))
```

;;;;;;;;;

será necesario modificar problema-busquedaA* para definir los estados inicial (A) y final (F). Finalmente ejecutamos (busqueda-A* problema-busquedaA*)

==> (ATOB BTOC CTOD DTOE ETOF)

así pues, está claro que se implementa la versión completa.

Recursos

Para realizar esta PEC el material imprescindible son los temas 2, 3 y 4 el Módulo 2.

Criterios de valoración

Problema 1: 20%

Problema 2: 20%

Problema 3: 40%

Problema 4: 20%

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC2 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **12 de Abril** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.